TECHNICAL REPORT

Survival Model Evaluation: flchain

Fitted with the survival-analysis-pipeline.

Source data	`datasets/flchain.csv.gz`
Rows	7,871 (27.5% with the ending observed, 72.5% censored)
Start dates	1995-01-01 to 2003-01-01
Evaluation	3 expanding-window temporal folds
Source of figures	`reports/flchain_demo/metrics.json`, regenerated by the command in section 9

---

1. Summary

This report evaluates two survival models fitted to `flchain.csv.gz`. It holds 7,871 rows, each observed from its start date. 27.5% have observed endings and 72.5% are censored, still running when observation stopped. Median observed duration, censored rows included, is 4303 days, the scale on which every horizon and prediction below sits. The models predict, from what was on file at the start date, how long each row survives.

When evaluating the two models' performance, the apples-to-apples comparison is the fold-mean concordance, the share of row pairs a ranking orders correctly where 0.500 is a coin flip. The pipeline uses walkforward validation. Each of the 3 temporal folds trains both models on one stretch of history and tests them on the next, and the 3 scores average. On concordance the XGBoost accelerated-failure-time (AFT) model scores 0.771 and the Cox proportional hazards baseline 0.795. The Cox baseline scores higher.

To attach an uncertainty range, which says how far the score could reasonably move, the AFT model is also scored with all 4,723 out-of-time test rows pooled into one list. That concordance is 0.750, with a 95% bootstrap interval of 0.735 to 0.764. Section 5 explains the two figures and how to read the interval.

The pooled score splits between `flc_band` group membership and ranking within it, where `flc_band` is the grouping column this run was given. Ranking rows by their group's average prediction alone scores 0.653, and comparing only rows inside the same group scores 0.696, clear of the coin flip. Section 5 gives the decomposition.

The boosted model's probabilities lose to a no-skill forecast at 365, 730, and 1460 days. A no-skill forecast assigns every row the same population-average probability. The limitations section says what remains usable.

Both models are saved in one bundle, which records the Cox baseline as recommended for scoring new rows.

2. Motivation

Analyst notes:

The serum free light chain cohort is one of the most widely taught survival datasets. It ships with R's survival package, it appears in the standard Python tutorials, and a reader who has studied survival analysis likely arrives already knowing roughly what a model scores on it. That makes it a useful public test. The pipeline's numbers can be checked against reported ones, and where they differ, the difference has to be explainable.

The data is also a hard test. Nearly three quarters of the subjects were still alive when the study stopped watching, so a model here must learn to predict lifetimes from lifetimes it mostly never saw end. How each of the two models holds up under that much censoring is the run's main finding, and the interpretation section reports it.

3. Data

Durations are measured in days. Columns dropped before fitting: flc_band.

Left truncation, where a row was already running when the source's records begin, leaves a recorded start that is not the true start. This pipeline has no delayed-entry handling, so it cannot take a row that entered observation partway through its life, and those rows must be excluded during preparation. Whether they were, and how, is recorded in the dataset's own documentation.

Analyst notes:

Each row is one subject in the Mayo Clinic serum free light chain study, observed from their blood sample until death or the end of follow-up. Only 28% of subjects died during follow-up. The features are: age at sampling, sex, the kappa and lambda free light chain concentrations, serum creatinine, and the assay decile group, the study's own one-to-ten grouping of the light-chain result. It also includes a flag for a prior diagnosis of MGUS, a benign precursor blood condition (monoclonal gammopathy of undetermined significance).

The recorded start date is a year, with no month or day. This complicates preparation. Two subjects sampled in the same year cannot be put in time order, and there are only a few distinct start dates to cut folds at. Asking for too many folds cut the same-year rows by their position in the file, so two folds could share a split date and differ only in which rows they held. Because the source file lists subjects in age order within each year, that cut gave different folds different age groups rather than a fair mix of ages. And because age is a major driver of survival, some folds scored far better than others. To prevent that, the preparation script shuffles within each year with a fixed seed. That makes the order genuinely arbitrary instead of the source

file's hidden age sort. The pipeline also merges any requested folds that land on the same split date into one. The interpretation section gives the numbers.

Survival time starts at the blood sample for every subject by construction, so no subject is first seen partway through the interval being measured, and the left-truncation fault described above does not arise in this dataset.

Figure 1 shows Kaplan-Meier survival curves by `flc_band`, the coarsest structure in the outcome before any model is fitted.

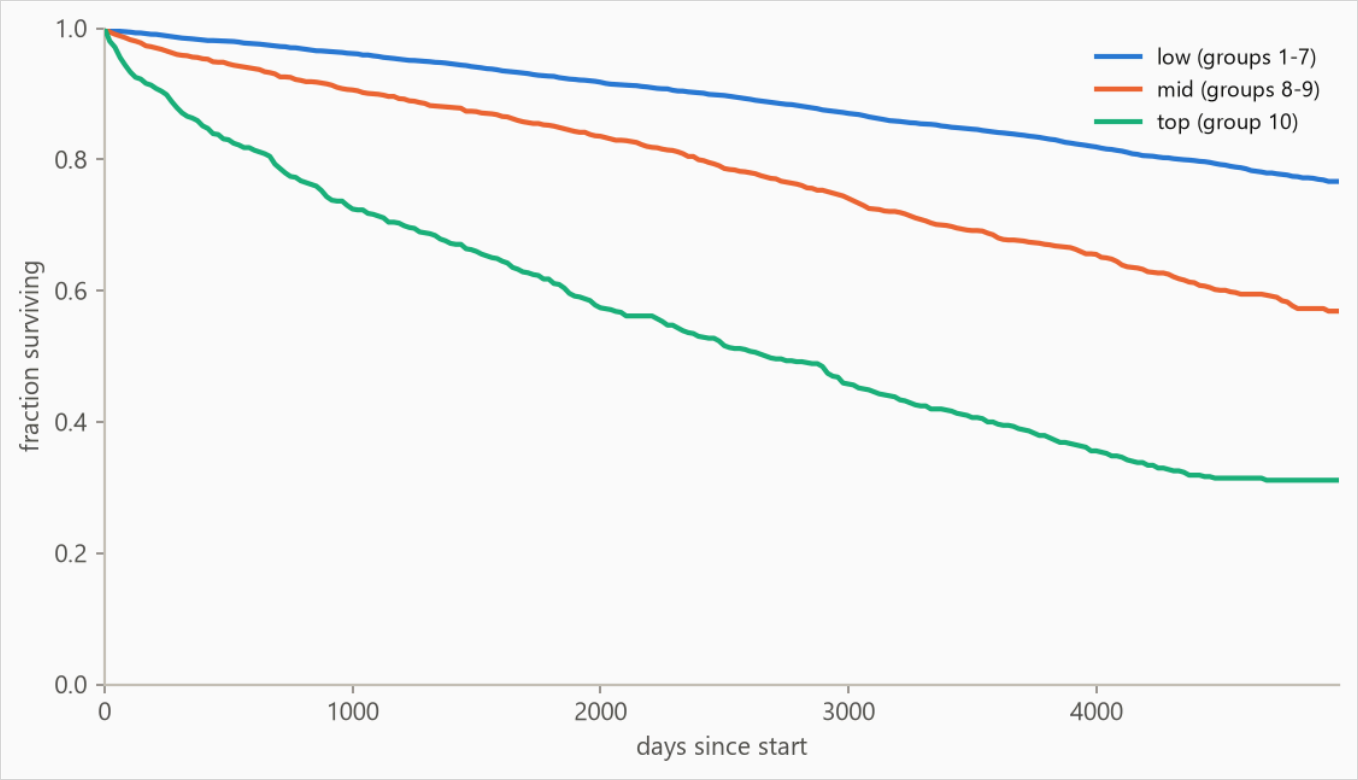


Figure 1. Kaplan-Meier survival curves by `flc_band`: the fraction of each group still running at each age, with censored rows counted for as long as they were observed. Groups beyond the seven most frequent, where present, are collapsed into (other) for this plot only.

4. Method

4.1 Model class

The pipeline fits two models on every run. The first is a boosted-tree accelerated failure time (AFT) model, XGBoost with its `survival:aft` objective, built for durations with incomplete observations. An observed ending tells the model the exact lifetime, and a censored row tells it only "at least this long", so every row

contributes. It predicts each row's median survival time in days, and a log-normal curve around that median, whose width is fitted once and shared by every row, gives the probability of surviving any horizon. The second is a Cox proportional hazards baseline, the standard linear survival model, fitted with lifelines' `CoxPHFitter` on the same features to answer whether the boosted model was necessary. Its coefficients are fitted under a penalty that pulls them toward no effect, which holds the fit steady when features move together but leaves the uncertainty intervals drawn around those coefficients approximate. The run recommends whichever scores the higher fold-mean concordance.

The two differ in what they assume. The AFT model assumes every row follows the same survival curve run on a faster or slower clock, so features change how fast that clock runs, never the curve's shape. The Cox model makes no assumption about that shape at all, estimating the curve from the data by counting who was still running at each age. In exchange it assumes each feature multiplies risk by the same factor at every age, which this report does not test. Which set of assumptions suits a dataset cannot be known in advance, which is why both are fitted and scored.

Numeric features pass through, missing values included (the Cox baseline gets train-window median imputation). Text columns are one-hot encoded with the vocabulary refit per training window, so a fold's features reflect only what was on file by its split date.

4.2 Temporal validation and label re-censoring

Evaluation uses 3 expanding-window folds ordered by start date. The earliest 40% of rows is set aside as burn-in and never tested. Each fold trains on every row started before its split date and tests on the next block. A split date can only fall on a date the data contains, so the first fold trains on 1,275 rows, the burn-in rounded to a date boundary (sizes in Table 2).

Training labels are re-censored at each split date. A row started long before a split may have died after it, and its label contains that future, so every post-split death is rewritten as a censoring at the split. Omitting this raises scores by importing the future.

4.3 Selection and calibration

The boosted model's hyperparameters are selected once on the first fold's training window by an inner temporal split, the same past-then-future cut made inside that window. Concordance grades only whether rows are ordered correctly, and a model can order them well while drawing survival curves far too wide or too narrow. So selection is scored on held-out censored log-likelihood instead, which grades the whole predicted distribution against what was observed.

The predictive scale is the width of the boosted model's log-normal curve, one unitless number shared by every row. A larger scale spreads the model's probability over a wider range of lifetimes. Two values of it

appear in this run. The first, 1.2, is the setting used while training, where it shapes how the medians are fitted. The second, 1.46, is measured afterwards. With the fitted medians held fixed, it is the width that best matches the held-out outcomes on the training window's most recent stretch, and it is carried to the model refitted on the full window. The training value answers which width trains the best medians. The measured value answers which width matches the outcomes those medians actually got. The two need not agree, and every probability the boosted model reports here uses the measured value.

5. Results

5.1 Discrimination

Table 1. Concordance index by model, pooled over 4,723 test rows with 95% percentile bootstrap intervals over 500 resamples. Higher is better, and 0.500 is a coin flip, and Harrell's C is the standard estimator of it. Fold-mean rows score each of the 3 folds separately and average. The interval resamples test rows with the fitted models held fixed, so it measures scoring precision, not stability across history. The fold spread is the guide to that.

METHOD	CONCORDANCE (HARRELL'S C)	95% INTERVAL
XGBoost AFT (pooled)	**0.750**	**0.735 to 0.764**
XGBoost AFT (fold mean)	0.771	not resampled
Cox proportional hazards (fold mean)	0.795	not resampled

Each fold fits its own models. Because the AFT model predicts a survival time in days, and days is a universal unit that exists outside of an individual fold model, its predictions can be pooled into one list while Cox proportional hazards models cannot. A Cox model predicts a hazard score, the risk of ending at any given moment, relative to the average row in the window it was trained on, not universal time units. As a result, Cox scores from different folds cannot be compared in one list. For a fair comparison, the two models must be compared on fold mean concordance, which is each fold scored on its own and the 3 scores averaged.

The pooled row answers a different question. It scores all test rows in one list, which is enough data to attach an uncertainty range as seen in Table 1. The interval is the range the score stayed inside for 95% of resamples of the test rows. A narrow interval means the score is precise on this test set. And an interval that sits wholly above 0.500 means the model beats a coin flip even at its low end.

It is important to determine whether the model does more than recognize which group a row belongs to. In order to assess this, we split the pooled concordance by `flc_band`. Re-ranking rows by their group's average replaces every row in a group with the same score, so comparisons only ever run between groups. Evaluating the pooled concordance on those re-ranked rows gives a score of 65.3%. The boosted model scores each row

individually instead, which orders rows inside a group. Inside a group this ranking is correct 69.6% of the time, averaged over the 3 groups large enough to score (at least 50 rows and 10 observed endings), each weighted by its number of comparable pairs. When the group-average score matches or beats the model's own pooled score, the model's ranking comes from telling groups apart. When the within-group score sits well above 0.500, the model also orders rows inside a group, which is the part a lookup table of group averages cannot do.

Figure 2 plots the per-fold concordance from Table 2.

Table 2. Per-fold results. Censoring is the share of each fold's test rows whose ending was not observed. The 5 requested folds merged to 3 where the start dates' granularity gave several the same split date.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX
1	1996-01-01	1,275	1,890	71%	0.734	0.797
2	1997-01-01	4,765	1,889	75%	0.777	0.785
3	1999-01-01	6,833	944	84%	0.803	0.803

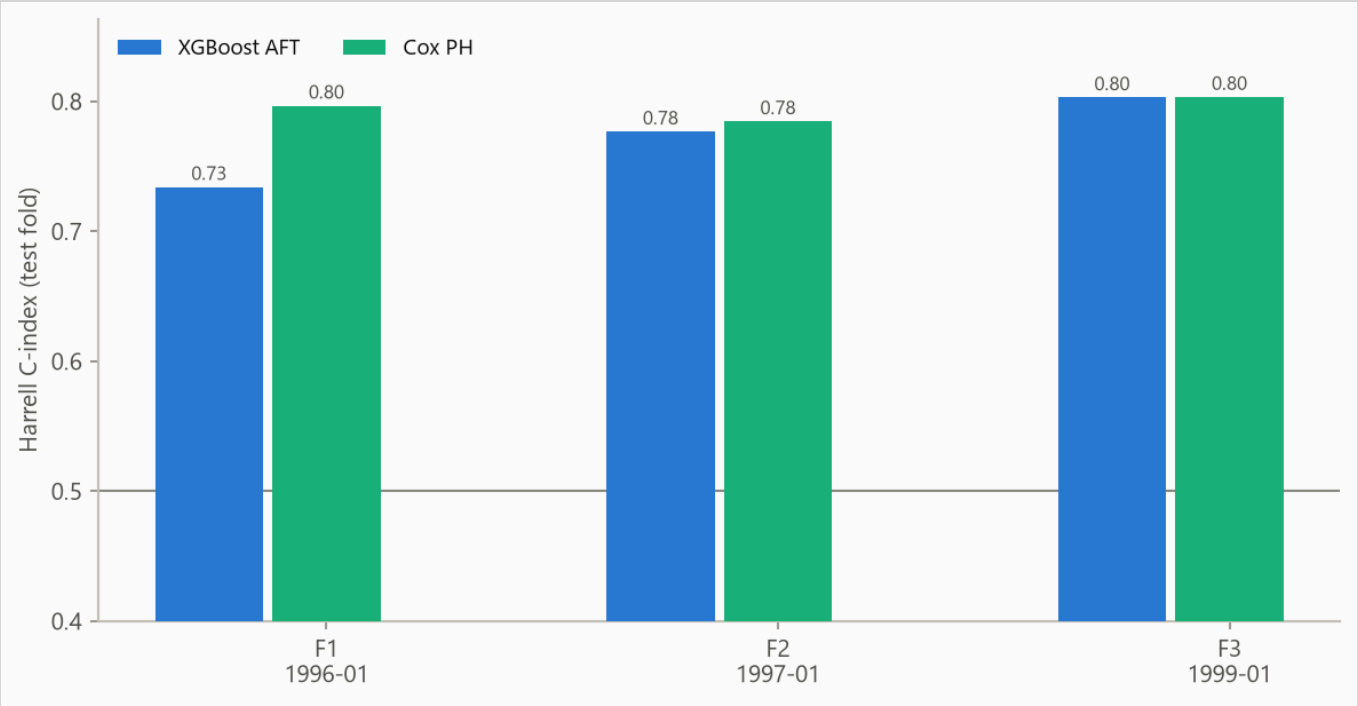


Figure 2. Concordance by fold. The boosted model spans 0.734 to 0.803. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

5.2 Calibration

A majority of this report focuses on concordance. But concordance grades only whether models produce the correct ordering when comparing two rows. It cannot be used to assess the absolute accuracy of the predicted probabilities. To assess the absolute accuracy, we use the Brier score, the mean squared error of a probability forecast (lower is better). Censored rows can bias the Brier score, so we use the inverse probability of censoring weighting (IPCW) to compensate. At each horizon in Table 3, the models predict each row's chance of still running that length of time. The no-skill forecast predicts the same chance for every row, the share of the whole population still running at that horizon, estimated with the Kaplan-Meier method so censored rows count for as long as they were observed.

Table 3. IPCW Brier score by horizon. Lower is better, and the no-skill column is the score to beat, so a model above it adds nothing at that horizon.

HORIZON	AFT	COX	NO-SKILL FORECAST
365 days	0.034	0.030	0.032
730 days	0.059	0.048	0.051
1460 days	0.107	0.081	0.086

Figure 3 plots predicted against observed survival at 730 days for both models. Points falling on both sides of the diagonal are noise. Points falling consistently on one side are bias in that direction.

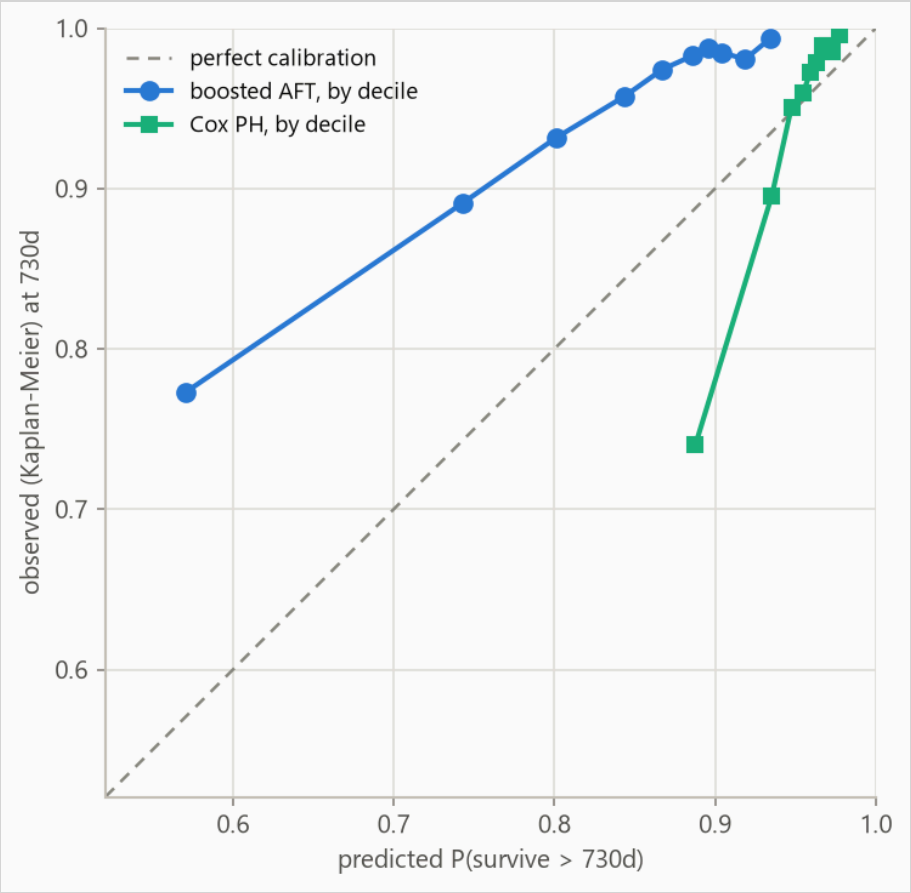


Figure 3. Predicted against observed survival at 730 days for both models, each binned on its own predicted deciles. Observed frequencies are Kaplan-Meier estimates within each bin, so censored rows contribute correctly. The largest deviation is 0.202 in the boosted model's decile 1 and 0.148 in the Cox baseline's decile 1. Deviations are probabilities. Deciles are cut on predicted value and hold 461 to 484 rows each. In small or heavily censored deciles the observed value carries more uncertainty than three decimals suggest.

6. Feature analysis

6.1 The boosted model

The scores above grade how well each model ranks rows and how accurate its probabilities are, not which inputs it used. A feature attribution answers that. For one row and one feature, it is that feature's contribution to the difference between that row's prediction and the model's average prediction. The method used here is SHAP (SHapley Additive exPlanations).

Attributions sit on the log scale of survival time, so an attribution is a multiplier on predicted time rather than a number of days. An attribution of +0.3 multiplies that row's predicted survival time by about 1.35, and a negative one shortens it. That conversion follows from the scale and says nothing about this dataset.

Averaging a feature's attributions by size, ignoring sign, says how much that feature moves predictions across the explanation sample, a random subset of the final model's training rows drawn for the attribution computation. A feature high in that average moves predictions a lot, and one low in it barely moves them.

Attributions are descriptive and in-sample, computed on the final boosted model's own training rows. Correlated features split credit by the model's internal choices as much as by the data, so directions are more trustworthy than magnitudes.

Figure 4 ranks the strongest features by that average, and Figure 5 shows, for each of those features, which direction it pushed and how much that varied from row to row.

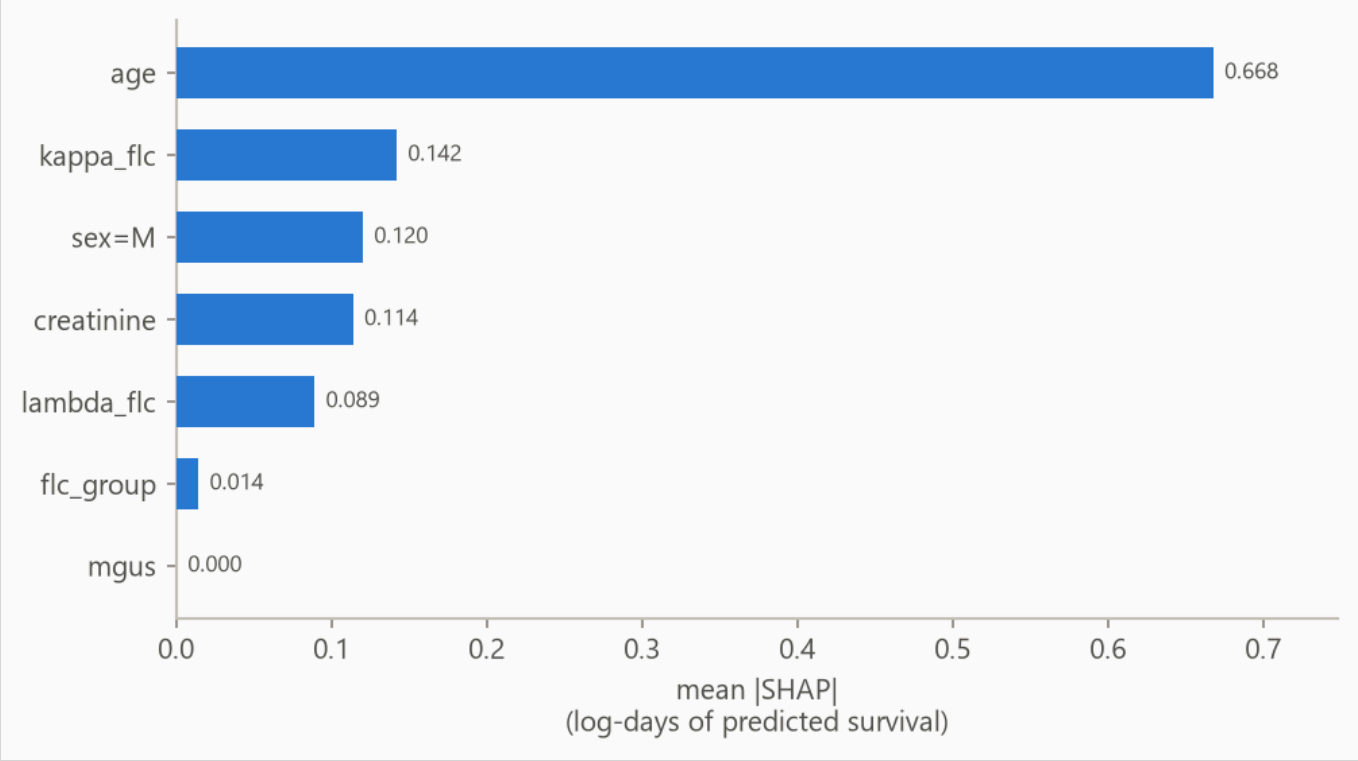


Figure 4. The strongest features, ranked by the average size of their attributions across the explanation sample. Size is taken without regard to sign, so this says how much each feature moves predictions and not which way it moves them; the direction is in Figure 5. The scale is the same log scale of survival time, so a feature averaging 0.10 moves predicted survival time by about a tenth on a typical row. A yes/no flag from a text column keeps its column name in the label. Features below the strongest few are not drawn.

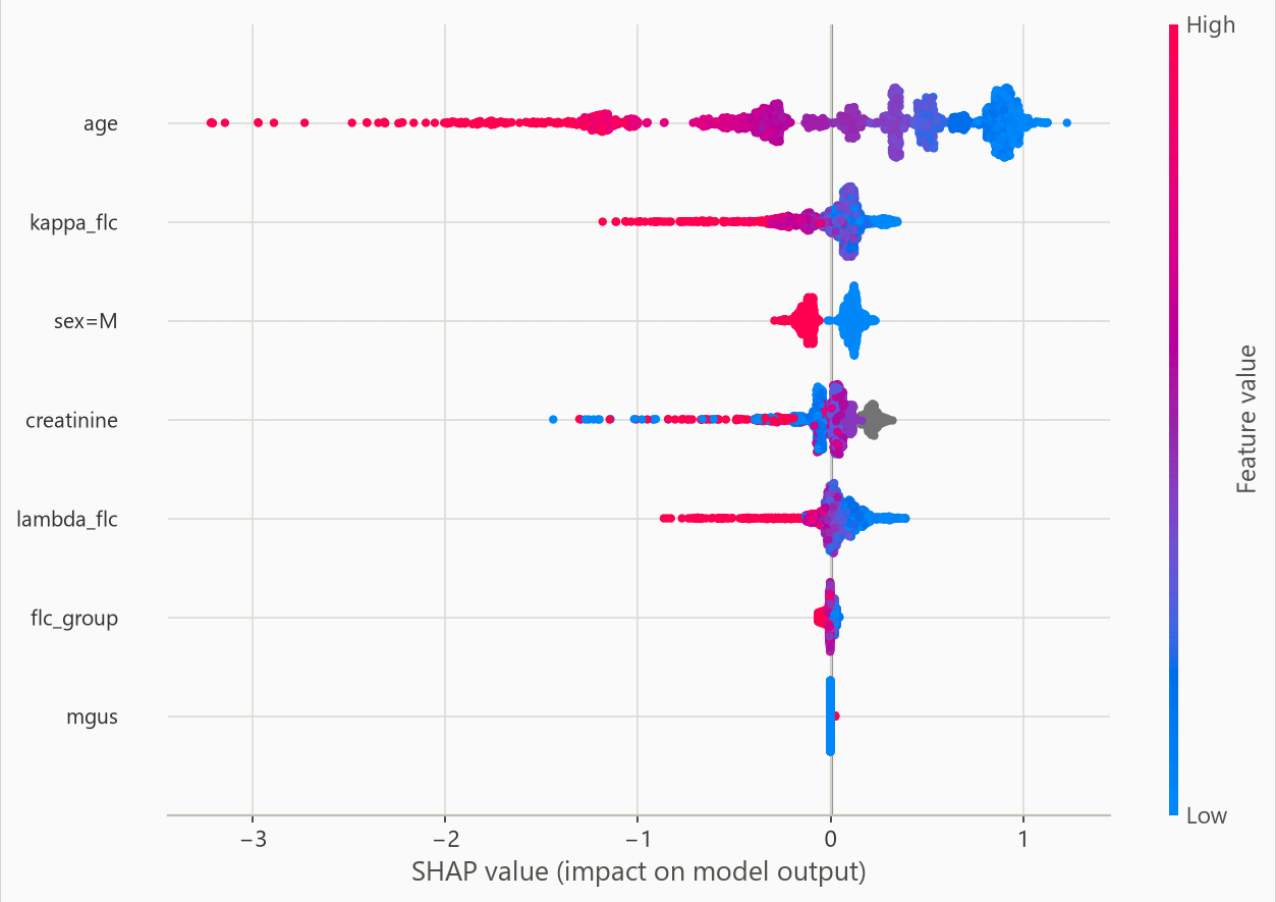


Figure 5. One row per feature, one dot per row of the explanation sample. The dot's position is the attribution that feature earned for that row, so dots left of zero shortened its predicted survival time and dots right of it lengthened. The dot's color is that row's value of the feature, red high and blue low, so a row whose reds sit left of its blues is a feature whose high values shorten survival. How far the dots spread says how much the feature's effect varies from row to row, and a feature pinned at zero did nothing. Single attributions run wider than the averages in Figure 4, since averaging over rows shrinks them. A yes/no flag from a text column keeps its column name in the label and reads the same way, with red meaning the row has that value.

6.2 The Cox baseline

The Cox baseline's drivers are its coefficients, reported as hazard ratios. A hazard ratio is the factor by which one unit of a feature multiplies the hazard and a hazard is the risk of an ending at a given age. The factor is the same at every age, so a ratio above 1 shortens survival. Figure 6 plots the strongest covariates.

Figure 4 describes the boosted model and Figure 6 the Cox baseline, two different models read by two different methods, so they need not name the same features, and neither is a conclusion about the data.

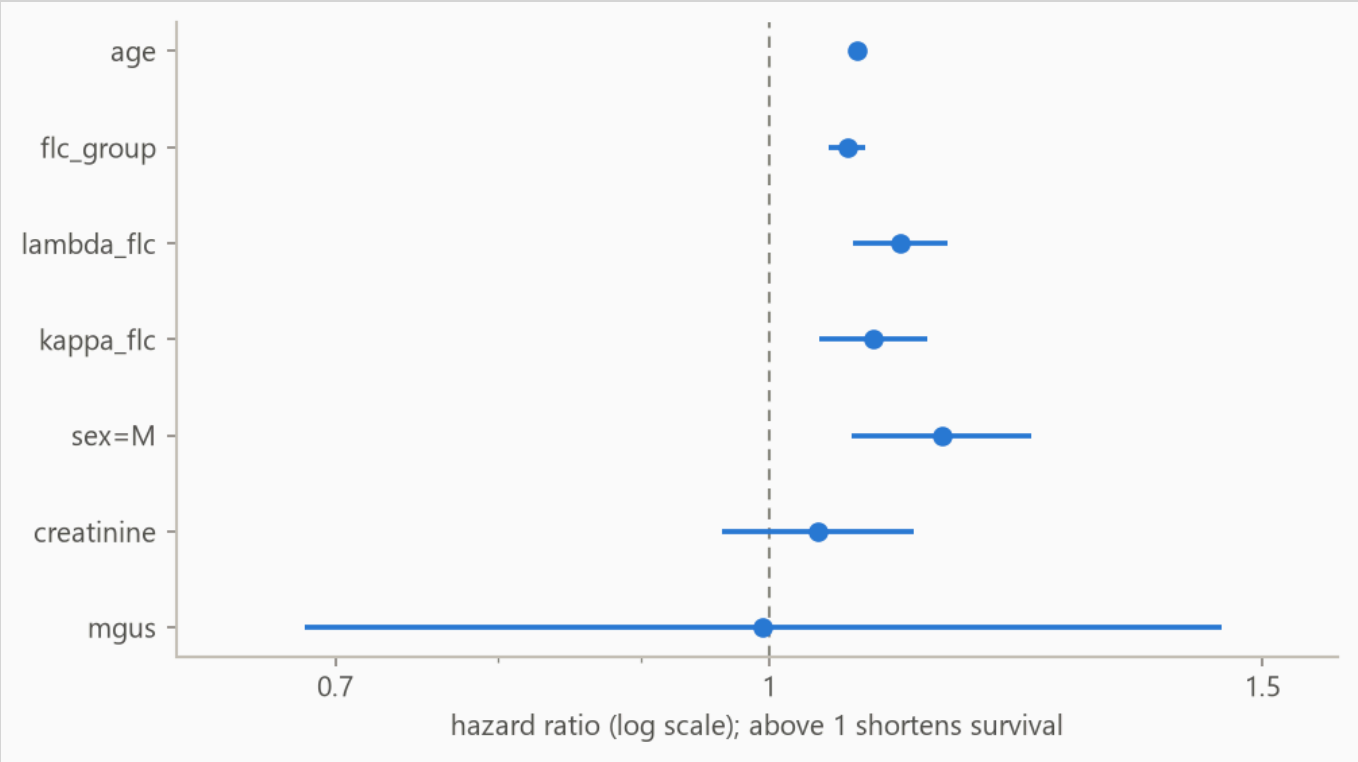


Figure 6. The Cox Proportional Hazards model's strongest covariates with their 95% intervals on a log axis. Dots right of the line shorten survival and left lengthen it. The covariate whose effect the data pins down most firmly sits at the top, and only the strongest few are drawn. Ratios for `sex` are relative to F, the reference level, chosen alphabetically and given no bar of its own.

7. Interpretation

Analyst notes:

How this compares to the standard exercise. This dataset is a teaching classic, and the usual exercise evaluates it with a random split. A random subset of people is hidden, the model trains on the rest, and its training pool therefore spans every year of the study. A benchmark paper (BigSurvSGD, arXiv:2003.00116, Table 2) reports a concordance of 0.792 for a standard Cox fit and 0.794 for its own method. It does not say how it held out its test data, so its figures cannot be tied to either protocol.

A random split suits the original research goal. A researcher asking whether the blood assay predicts mortality beyond age wants to know whether the relationship is real. The whole completed study is fair evidence for that, and the deliverable is the finding itself. Training only on the past suits the deployment goal. A clinic scoring each new patient at intake stands at a moment in time and knows only what has happened so far, and a test is only fair if it rehearses that position. This pipeline runs the second protocol. The Cox baseline's fold mean is 0.795, close to the published figures, with the three folds at 0.797, 0.785, and 0.803.

What an earlier version got wrong. An earlier version of this run reported a different result, and what retired it is worth recording. The source file lists subjects within each sample year in age order. Start dates here are year-only, so a fold boundary that lands inside a year cannot cut on time and cuts between rows instead, and row order here was age order. Two folds trained on identical windows and scored 0.649 and 0.847 on the unshuffled file, purely from who landed in their test slices. The preparation script now shuffles within each year with a fixed seed, folds that snap to the same split date merge into one, and the horizons now sit within the longest training window's observed follow-up. A random split can never meet this failure, because it never cuts along the file at all. A temporal protocol on coarse dates has to, which is why the run documents it.

Why the boosted model still trails. Only 28% of subjects died while the study watched. Under that much censoring the boosted model trails the Cox baseline on ranking, 0.771 against 0.795 on the fold mean, and on probability quality, running behind the no-skill forecast at every horizon while Cox runs slightly ahead. The Brier table in the results section grades this, and lower is better. The boosted model fits a fixed curve shape and guesses lifetimes from mostly unfinished ones. Cox reads its curve off the data by counting who is still alive at each age, which holds up where the data can check it. This is why the pipeline fits both models and lets the data choose, and this run's bundle recommends Cox.

What the attribution shows. Age carries most of the ranking, and the two light-chain measurements add signal on top, which is the original study's own finding. The decomposition by the assay's banding makes the model's contribution precise. A low, middle, and top banding of the ten assay groups alone ranks survival at 0.653, and comparisons restricted to subjects within the same band score 0.696, so the model adds real ranking beyond the assay's own grouping. The Kaplan-Meier figure in the data section shows the banding's separation directly.

8. Limitations

The boosted model's absolute probabilities are not usable at 365, 730, and 1460 days. The boosted model loses to the no-skill forecast on the censoring-weighted Brier score there, and Table 3 grades each model separately. Ranking and probability quality are separate, so ordering rows is still supported even at the horizons where the probabilities lose.

Censoring may be informative. The evaluation assumes rows stop being observed for reasons unrelated to their risk. If the rows that drop out of observation are the ones about to end, the survival estimates come out too high.

Analyst notes:

Folds merge where the dates cannot separate them. Start dates are years, so several requested folds can snap to the same split date. Those merge into one fold with the combined test block rather than posing as

distinct models, which is why the report shows three folds against the five the reproduce command requests.

Year-floored dates leak a little future into training labels. Every start is recorded as January 1 of its sample year, backdating subjects by up to a year. Re-censoring therefore keeps some deaths that truly happened just after a split inside that split's training labels. The bias runs toward optimism, is bounded by the deaths falling within a year of each split, and finer dates would remove it. Its size is not measured here.

Horizons stay inside the longest window's observation. The training windows watch subjects for one to four years, so the report grades probabilities at one, two, and four years only. An earlier version of this run asked these windows for ten-year probabilities and got extrapolations dressed as measurements. Support is still per fold. The one-year row is inside every window's observation, while the four-year row leans on the widest window, and the earlier folds contribute extrapolated predictions there. The Brier rows in the results section grade all of it, and lower is better there.

The boosted model gives every row one curve shape. The predictive scale is a single fitted number, so the model runs a row's clock faster or slower but never changes the curve's shape. Per-row width is future work.

Harrell's concordance is biased under heavy censoring, usually upward. It scores only the pairs where the earlier ending is known, and under heavy censoring those pairs over-represent short observed lifetimes. The most censored test window is 84% censored, so read its rows in Table 2 with the most doubt.

9. Reproducing this run

Python 3.11 or later. From the repository root:

```
python scripts/run_fit_evaluate.py --data datasets/flchain.csv.gz --name flchain --id-col subject_
python scripts/run_build_report.py --run reports/flchain_demo
```

Every number is read from the run's `metrics.json` at build time, and a figure the prose never cites fails the build. `requirements.txt` pins the exact versions the committed numbers came from.

Generated from `reports/flchain_demo/metrics.json` by `scripts/run_build_report.py --run`. 7,871 rows, 4,723 out-of-time test rows.